Web Programming

Kiarash Sanei

June, 2026

Contents

1 Introduction to Web Programming **2**
- 1.1 Brief History of the Web . . . 2
- 1.2 Internet vs. the WWW . . . 2
- 1.3 Web Architecture Evolution . . . 2
- 1.4 The Layers of the Web . . . 2
- 1.5 The Holy Trinity of the Web . . . 2
- 1.6 Technology Stacks . . . 2
- 1.7 Developer Roles . . . 3
- 1.8 Software Engineering Principles . . . 3
- 1.9 LLM Best Practices . . . 3

2 HTML — The Building Blocks of the Web **4**
- 2.1 What is HTML? . . . 4
- 2.2 HTML Document Structure . . . 4
- 2.3 HTML Elements . . . 4
- 2.4 The `<head>` Element . . . 4
- 2.5 Text Elements . . . 5
- 2.6 Hyperlinks . . . 5
- 2.7 List Elements . . . 6
- 2.8 Media Elements . . . 6
- 2.9 Tables . . . 6
- 2.10 Forms . . . 7
- 2.11 Identifying Elements: `id` and `class` . . . 7
- 2.12 Sectioning the Page . . . 8
  - 2.12.1 Using `<div>` . . . 8
  - 2.12.2 Using Semantic Tags (Preferred) . . . 8
- 2.13 Inline vs. Block Elements . . . 8
- 2.14 Comments . . . 8
- 2.15 Debugging HTML . . . 8

3 CSS — Pages That Look Elegant **9**
3.1 What is CSS? . . . 9
3.2 CSS Rule Syntax . . . 9
3.3 CSS Selectors . . . 9
3.4 Pseudo-class Selectors . . . 9
3.5 Attribute Selectors . . . 9
3.6 Element Selectors (Type Selectors) . . . 10
3.7 Class Selectors . . . 10
3.8 Pseudo-element Selectors . . . 10
3.9 Including CSS in HTML . . . 10
3.10 CSS Specificity and the Cascade . . . 10
3.11 CSS Measurement Units . . . 11
3.12 Text and Font Properties . . . 11
3.13 Color Properties . . . 11
3.14 The CSS Box Model . . . 11
3.15 Borders and Outlines . . . 12
3.16 CSS Display and Layout . . . 12
3.17 CSS Floats . . . 12
3.18 Flexbox . . . 12
3.19 CSS Grid . . . 13
3.20 CSS Positioning . . . 13
3.21 Responsive Design and Media Queries . . . 13
3.22 CSS Variables (`:root`) . . . 14

4 JavaScript Fundamentals **15**
4.1 Variables and Data Types . . . 15
4.2 Type Coercion . . . 15
4.3 Functions . . . 15
4.3.1 Hoisting . . . 15
4.3.2 Function Types . . . 16
4.3.3 Parameters . . . 16
4.3.4 Higher-Order Functions . . . 17
4.3.5 Immediately Invoked Function Expressions (IIFE) . . . 17
4.4 Objects and JSON . . . 18
4.5 Arrays . . . 18
4.6 The `this` Keyword . . . 18
4.7 Closures . . . 19
4.8 The Event Loop and Asynchronous JavaScript . . . 19
4.9 Callbacks . . . 19
4.10 Promises . . . 20

4.11 Async / Await 20
4.12 ES Modules 21

5 JavaScript Beyond the Browser **22**
5.1 Limitations of Plain JavaScript 22
5.2 Node.js 22
5.3 NPM (Node Package Manager) 22
5.4 Popular npm Packages 22
5.5 Bundling for Browsers 23
5.6 TypeScript 23
5.6.1 Motivation 23
5.6.2 What is TypeScript? 23
5.6.3 Variables and Types 23
5.6.4 Interfaces 24
5.6.5 Classes 24
5.6.6 Generics 24
5.6.7 Modules 25

6 The DOM and Browser APIs **26**
6.1 From Markup to DOM 26
6.2 DOM Structure 26
6.3 Querying DOM Elements 26
6.4 Events 26
6.5 Manipulating the DOM 27
6.5.1 Modifying Content and Styles 27
6.5.2 Adding and Removing Elements 27
6.5.3 Working with Classes 27
6.6 Forms and DOM 27
6.7 The `window` Object 28
6.8 Web Storage 28
6.8.1 `localStorage` 28
6.8.2 `sessionStorage` 28

7 React — UIs That React **29**
7.1 Motivation: Problems with Classic Frontend Development 29
7.2 The Rise and Limits of jQuery 29
7.3 Library vs. Framework 29
7.4 What is React? 29
7.5 Setting Up a React Project with Vite 29
7.6 JSX 30
7.7 React Components 30

7.8 Props . . . 31
7.9 State and `useState` . . . 31
7.10 Event Handling in React . . . 31
7.11 Lists in JSX . . . 32
7.12 The Virtual DOM and Reconciliation . . . 32
7.13 Component Composition . . . 32
7.14 React Developer Tools . . . 32
7.15 Styling in React . . . 32

8 React — Act II: Hooks, Lifecycle, and Patterns 33
8.1 Component Lifecycle . . . 33
8.2 The `useEffect` Hook . . . 33
8.2.1 Common `useEffect` Mistakes . . . 33
8.3 Fetching Data . . . 33
8.3.1 Using `fetch` . . . 33
8.3.2 Using Axios . . . 34
8.4 State Management Patterns . . . 34
8.4.1 Pattern I: Lifting State Up . . . 34
8.4.2 Pattern II: Callbacks as Props . . . 34
8.4.3 Pattern III: Context API . . . 35
8.5 Redux . . . 35
8.6 React Router . . . 35
8.7 Forms in React . . . 36
8.8 Error Handling in React . . . 37
8.9 shadcn/ui . . . 37
8.10 TanStack Query . . . 38
8.11 TanStack Mutations . . . 38
8.12 TanStack Best Practices . . . 38
8.13 Next.js and SSR . . . 39
8.14 Recommended Project Structure . . . 39

9 Quick Reference 40
9.1 JavaScript Cheat Sheet . . . 40
9.2 HTML Cheat Sheet . . . 40
9.3 CSS Cheat Sheet . . . 40
9.4 React Hooks Summary . . . 40

1 Introduction to Web Programming

1.1 Brief History of the Web

- The internet began as **ARPANET**, a US Department of Defense network (1960s–70s).
- Initial services: electronic mail and file transfer.
- Opened to commercial interests and most universities in the late 1980s.
- **WWW** (World Wide Web) created in 1989–91 by **Tim Berners-Lee**.
- Early browsers: Mosaic (1992), Netscape (1994), Internet Explorer (1995).
- Amazon.com opened in 1995; Google launched in January 1996.

1.2 Internet vs. the WWW

- **The Internet**: A network of networks using the Internet Protocol suite to link billions of devices worldwide.
- Consists of millions of private, public, academic, business, and government networks.
- Networks are linked by electronic, wireless, and optical networking technologies.
- Carries services such as the WWW, email, FTP, streaming, etc.
- **The WWW**: A system of interlinked hypertext documents accessed via the internet using HTTP.

1.3 Web Architecture Evolution

- **1990s**: Static websites — simple HTML pages served directly from a web server.
- **Early 2000s**: Dynamic web — server-side rendering with PHP, ASP, and JSP.
- **Mid-2000s**: MVC frameworks — Rails, Django, and Spring MVC for cleaner app design.
- **2010s**: Single-Page Apps (SPA) — React, Angular, and Vue move logic to the client side.
- **Late 2010s**: Serverless & Microservices — apps split into small, independent services.
- **2020s**: Modern hybrids — Next.js, Remix, and Astro blend SSR, SPA, and static generation.

1.4 The Layers of the Web

$$\texttt{Client (Front-end)} \xrightarrow{\text{HTTP Request}} \texttt{Network} \xrightarrow{\text{HTTP Response}} \texttt{Server (Back-end)}$$

- The **front-end** handles the user interface and runs in the browser.
- The **back-end** handles business logic, databases, and API endpoints.
- Communication happens over **HTTP** using URLs and API endpoints.

1.5 The Holy Trinity of the Web

- **HTML** — Structure (what the content *is*).
- **CSS** — Style (how it *looks*).
- **JavaScript** — Behaviour (how it *reacts*).

1.6 Technology Stacks

- A **stack** is the set of technologies used to build a web application.
- Combines frontend, backend, database, and infrastructure layers.
- Each layer handles a different part of the application (UI, logic, data, hosting).
- Choosing the right stack affects performance, scalability, and developer workflow.
- **Full-stack developers** work across all layers.

1.7 Developer Roles

- **Front-End Developer**: Builds interactive, user-friendly web interfaces using HTML, CSS, JavaScript, and frameworks like React or Vue.
- **Back-End Developer**: Develops server-side logic, APIs, and database integrations using Python, Node.js, Golang, or .NET.
- **Full-Stack Developer**: Combines front-end and back-end skills to deliver complete web solutions.
- **Web DevOps & Cloud Engineer**: Manages deployments, CI/CD pipelines, and scalable cloud hosting.

1.8 Software Engineering Principles

- **Clean Code**: Ensures readability, consistency, and collaboration across teams.
- **Maintainable Code**: Easier debugging, feature additions, and long-term scalability.
- **Dead Code Removal**: Eliminates unused, redundant logic to reduce complexity.
- **Fail Fast Approach**: Detects errors early, improving reliability and lowering costs.

1.9 LLM Best Practices

- **Define clear goals**: Know exactly what you want the model to achieve before prompting.
- **Prompt precisely**: Use clear, specific, and contextual instructions.
- **Iterate & refine**: Test, review, and adjust prompts or parameters for consistency.
- **Keep context short & relevant**: Provide only necessary information to reduce confusion.
- **Validate outputs**: Always fact-check and post-process model responses.
- **Respect safety & ethics**: Ensure responsible AI use.
- Use of LLMs in this course is permitted **as long as you understand the code**.

2 HTML — The Building Blocks of the Web

2.1 What is HTML?

- **HTML** stands for **HyperText Markup Language**.
- Used to **structure and describe** the content of a webpage.
- Consists of **elements** that describe different types of content.
- Web browsers understand HTML and render it as websites.
- **HTML is not a programming language** — it is a markup language.

2.2 HTML Document Structure

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>My Page</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <h1>Hello, World!</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

Listing 1: Basic HTML5 document structure

- `<!DOCTYPE html>` declares the HTML version (HTML5).
- `<html>` wraps the entire document.
- `<head>` contains meta information (not displayed directly).
- `<body>` holds all visible content.
- Tags are structured hierarchically and must be properly opened and closed.

2.3 HTML Elements

An HTML element typically has:

- An **opening tag**: `<button>`
- A **closing tag**: `</button>`
- **Attributes** (key-value pairs): `id="submit"`
- **Child elements or content** inside the tags.

2.4 The `<head>` Element

- Contains information *about* the page, not displayed directly.
- Helps browsers, search engines, and devices understand your page.
- `<title>` — sets the page title (shown on the browser tab).
- `<meta>` — defines metadata such as charset, author, viewport, and description.
- `<link>` — connects external resources like stylesheets or favicons.
- `<script>` — links or embeds JavaScript files.

2.5 Text Elements

- `<p>` — paragraph text.
- `<h1>` through `<h6>` — headings (largest to smallest).
- `<strong>` or `<b>` — bold font (`<strong>` carries semantic emphasis).
- `<em>` or `<i>` — italic font (`<em>` carries semantic emphasis).
- `<code>` — displays a piece of computer code inline.
- `<pre>` — preserves whitespace and displays preformatted text (often wraps `<code>`).
- `<span>` — generic inline container for styling parts of text.
- `<blockquote>` — block-level quotation.
- `<br>` — line break (void element, no closing tag).
- `<hr>` — horizontal rule / thematic break (void element, no closing tag).

```
<h1>Main Title</h1>
<h2>Subtitle</h2>
<p>This is a <strong>bold</strong> and <em>italic</em> paragraph.</p>
<p>Inline code: <code>console.log("hello")</code></p>
<pre><code>
function greet(name) {
  return "Hello, " + name;
}
</code></pre>
```

Listing 2: Text element examples

2.6 Hyperlinks

- Created using the `<a>` (anchor) tag.
- The `href` attribute specifies the URL or location to link to.
- `target="_blank"` opens a link in a new tab.
- The `download` attribute triggers a file download.
- **Internal links**: `href="/about"`
- **External links**: `href="https://wikipedia.org"`
- **Anchor links**: `href="#section-id"` (jumps to a section on the same page)
- **Mail links**: `href="mailto:info@example.com"`
- **Tel links**: `href="tel:+9821000000"`

```
<a href="https://google.com" target="_blank">Visit Google</a>
<a href="/contact">Contact Us</a>
<a href="#top">Back to Top</a>
<a href="mailto:hello@example.com">Email Us</a>
<a href="report.pdf" download>Download Report</a>
```

Listing 3: Hyperlink examples

2.7 List Elements

- `<ul>` — unordered list (bulleted).
- `<ol>` — ordered list (numbered).
- `<li>` — list item (used inside both `<ul>` and `<ol>`).
- `<dl>`, `<dt>`, `<dd>` — description list with terms and definitions.

```
<ul>
  <li>Apple</li>
  <li>Banana</li>
</ul>

<ol>
  <li>Step one</li>
  <li>Step two</li>
</ol>

<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language</dd>
</dl>
```

Listing 4: List examples

2.8 Media Elements

- `<img src="..." alt="...">` — embed an image (`alt` is required for accessibility).
- `<audio controls>` — embed an audio player.
- `<video controls width="...">` — embed a video player.
- `<source>` — specifies multiple media sources inside `<audio>` or `<video>`.
- `<iframe>` — embeds another webpage or external content.

```
<img src="photo.jpg" alt="A landscape photo" width="600" />

<audio controls>
  <source src="song.mp3" type="audio/mpeg" />
  Your browser does not support audio.
</audio>

<video controls width="640">
  <source src="clip.mp4" type="video/mp4" />
  Your browser does not support video.
</video>
```

Listing 5: Media elements

2.9 Tables

- `<table>` — defines the table.
- `<tr>` — table row.
- `<th>` — header cell (bold by default).
- `<td>` — data cell.
- `<thead>`, `<tbody>`, `<tfoot>` — optional sections for organizing large tables.
- Use `colspan` and `rowspan` attributes to merge cells.

```html
<table border="1">
  <thead>
    <tr>
      <th>Name</th>
      <th>Age</th>
      <th colspan="2">Contact</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Alice</td>
      <td>22</td>
      <td>alice@mail.com</td>
      <td>+1-555-0100</td>
    </tr>
  </tbody>
</table>
```

Listing 6: Table example with merged cell

2.10 Forms

- `<form>` — wraps all form controls; has `action` (where to send) and `method` (`GET`/`POST`).
- `<input>` — a variety of controls selected via the `type` attribute.
- `<textarea>` — multi-line text input.
- `<select>` + `<option>` — dropdown menu.
- `<button>` — clickable button.
- `<label>` — associates descriptive text with a control (use `for` attribute).
- `<fieldset>` + `<legend>` — groups related controls.

```html
<form action="/submit" method="POST">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" required />

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" />

  <label for="role">Role:</label>
  <select id="role" name="role">
    <option value="dev">Developer</option>
    <option value="design">Designer</option>
  </select>

  <label>
    <input type="checkbox" name="agree" /> I agree to the terms
  </label>

  <button type="submit">Send</button>
</form>
```

Listing 7: Sample HTML form

2.11 Identifying Elements: `id` and `class`

- `id` — a **unique** identifier for a single element on the page.
- `class` — groups multiple elements with shared styling or behaviour.
- An element can have only **one** `id`, but **multiple** class values (space-separated).

- `id` is used for anchors, scripting, or targeting specific elements.
- `class` is used mostly for styling and reusable components.

2.12 Sectioning the Page

2.12.1 Using `<div>`

- A generic container to group HTML elements.
- Does not carry any semantic meaning on its own.
- Commonly used with `class` or `id` to apply styles or behaviour.

2.12.2 Using Semantic Tags (Preferred)

- `<header>` — introductory content or navigation links.
- `<nav>` — major navigation block.
- `<main>` — dominant content of the `<body>`.
- `<section>` — thematic grouping of content.
- `<article>` — self-contained composition (blog post, news story).
- `<aside>` — tangentially related content (sidebar).
- `<footer>` — footer for the page or a section.

Semantic tags improve readability, accessibility, and SEO compared to using `<div>` everywhere.

2.13 Inline vs. Block Elements

- **Block elements**: Start on a new line and take up the full width available (e.g. `<div>`, `<p>`, `<h1>`, `<ul>`).
- **Inline elements**: Do not start on a new line; take only as much width as needed (e.g. `<span>`, `<a>`, `<strong>`, `<img>`).
- Inline elements **cannot** contain block-level elements.

2.14 Comments

```
<!-- This is a comment and will not appear in the browser -->
<p>This text is visible.</p>
```

Listing 8: HTML comment syntax

2.15 Debugging HTML

- Use the browser's **Inspect Element** (DevTools) to analyze structure, debug layout, and experiment live.
- Always look for the **root cause** of a problem, not just the symptom.
- *"Fix the problem, not the blame."* — The Pragmatic Programmer.
- The first rule of debugging: **Don't Panic**.
- Explaining a bug to someone else ("rubber duck debugging") often reveals the issue.

3 CSS — Pages That Look Elegant

3.1 What is CSS?

- **CSS** stands for **Cascading Style Sheets**.
- Used to **style** HTML elements — controls colors, fonts, layout, and animations.
- CSS **separates content** (HTML) from **presentation** (design).
- Helps create responsive, modern, and user-friendly designs.

3.2 CSS Rule Syntax

```
selector {
  property: value;
}

/* Example */
p {
  color: white;
  font-size: 16px;
}
```

Listing 9: CSS rule anatomy

3.3 CSS Selectors

- `element` — targets all elements of a type (e.g. `p`, `div`).
- `.class` — targets elements with a specific class.
- `#id` — targets a specific element with an ID.
- `*` — targets all elements (universal selector).
- `A, B` — targets both A and B elements.
- `A B` — targets B that is a **descendant** of A.
- `A > B` — targets B that is a **direct child** of A.
- `A + B` — targets B that is the **adjacent sibling** immediately after A.
- `A ~ B` — targets all siblings B after A.

3.4 Pseudo-class Selectors

- `:hover` — when the mouse is over the element.
- `:focus` — when the element is focused (e.g. an input field).
- `:first-child` / `:last-child` — first or last child of its parent.
- `:nth-child(n)` — every nth child (e.g. `:nth-child(2n)` for even rows).
- `:not(.active)` — all elements except those with `.active`.
- `:checked`, `:disabled`, `:placeholder-shown` — form-state selectors.

3.5 Attribute Selectors

- `[type]` — elements that have the `type` attribute.
- `[type="text"]` — elements where `type` equals `"text"`.
- `[href^="https"]` — `href` starting with `"https"`.
- `[href$=".pdf"]` — `href` ending with `".pdf"`.
- `[data-role*="btn"]` — attribute containing `"btn"`.

3.6 Element Selectors (Type Selectors)

- `p` — selects all paragraphs.
- `div` — selects all `<div>` containers.
- `h1`, `h2`, `h3` — selects all headings of a specific level.
- `ul li` — descendant combination: all `<li>` inside any `<ul>`.

3.7 Class Selectors

- `.classname` — selects all elements that have `class="classname"`.
- `div.classname` — selects only `<div>` elements with that class.
- `.class1.class2` — selects elements that have **both** classes.
- `.parent .child` — descendant combination with classes.

3.8 Pseudo-element Selectors

- `::before` — inserts content before the element's real content (requires `content`).
- `::after` — inserts content after the element's real content (requires `content`).
- `::first-line` — styles the first line of a block of text.
- `::first-letter` — styles the first character of a block of text.
- `::selection` — styles the part of the text that the user highlights.
- `::marker` — styles the bullet or number in list items.
- `::placeholder` — styles the placeholder text inside an `<input>` or `<textarea>`.

3.9 Including CSS in HTML

- **Inline CSS**: `<p style="color:red;">Hello</p>` — highest specificity, not recommended for large projects.
- **Internal CSS**: `<style>` block inside `<head>` — good for single-page documents.
- **External CSS**: `<link rel="stylesheet" href="style.css">` — best practice; styles apply to multiple pages.

```
<head>
  <link rel="stylesheet" href="styles.css" />
</head>
```

Listing 10: Linking an external stylesheet

3.10 CSS Specificity and the Cascade

Specificity determines which CSS rule wins when multiple rules target the same element.

1. **Inline styles** — highest priority.
2. **ID selectors** (`#id`) — very high.
3. **Class, pseudo-class, attribute selectors** — medium.
4. **Element and pseudo-element selectors** — lowest.

- When specificity is **equal**, the **last rule declared** wins.
- `!important` overrides all other rules (use sparingly).

3.11 CSS Measurement Units

- **Absolute units**: `px` (pixels), `pt` (points), `cm`, `mm`, `in`.
- **Relative units**:
  - `em` — relative to the **element's** font-size.
  - `rem` — relative to the **root** element's font-size.
  - `vw` / `vh` — 1% of the viewport width / height.
  - `%` — relative to the **parent** element.
  - `ch` — width of the "0" character; useful for text containers.

3.12 Text and Font Properties

```
p {
  font-family: 'Inter', Arial, sans-serif;
  font-size: 1rem;          /* 16px by default */
  font-weight: 600;         /* normal | bold | 100-900 */
  line-height: 1.6;
  text-align: justify;      /* left | center | right | justify */
  text-decoration: none;    /* underline | line-through */
  letter-spacing: 0.05em;
  text-transform: uppercase;
}
```

Listing 11: Common text properties

3.13 Color Properties

- `color` — sets the text color.
- `background-color` — sets the element background.
- `background-image` — sets a background image (`url(...)`).
- Colors can be expressed as:
  - Named: `red`, `cornflowerblue`
  - HEX: `#ff4455`
  - RGB / RGBA: `rgb(255, 0, 0)` / `rgba(255, 0, 0, 0.5)`
  - HSL / HSLA: `hsl(0, 100%, 50%)`
- Use high contrast for readability and accessibility.

3.14 The CSS Box Model

Every element is a rectangular box composed of (inside-out):

$$\text{content} \rightarrow \text{padding} \rightarrow \text{border} \rightarrow \text{margin}$$

- `width` / `height` — size of the content area.
- `padding` — space inside the border, around content.
- `border` — line surrounding the padding.
- `margin` — space **outside** the border (collapses between vertical siblings).
- `box-sizing: border-box` — makes `width`/`height` include padding and border (recommended).

```
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
```

Listing 12: Box model reset (best practice)

3.15 Borders and Outlines

- `border`: shorthand — `2px solid #fff`.
- Border styles: `solid`, `dashed`, `dotted`, `double`, `none`.
- `border-radius` — rounds corners; `50%` creates a circle.
- `outline` — drawn outside the border; **does not affect layout** (no space taken).

3.16 CSS Display and Layout

- `display: block` — element starts on a new line and fills available width.
- `display: inline` — flows with surrounding text.
- `display: inline-block` — flows inline but respects width/height.
- `display: none` — removes element from layout entirely.
- `display: flex` — activates Flexbox on the container.
- `display: grid` — activates CSS Grid on the container.
- `visibility: hidden` — hides element but **keeps its space**.

3.17 CSS Floats

- `float: left | right` — element floats to the side; surrounding text wraps around it.
- `clear: left | right | both` — prevents the following element from wrapping around floats.
- Mostly superseded by Flexbox and Grid for layout, but still used for text-wrapping around images.

3.18 Flexbox

- A **one-dimensional** layout method (one row *or* one column at a time).
- `display: flex` — makes the element a flex **container**; direct children become flex **items**.
- `flex-direction: row | column | row-reverse | column-reverse`.
- `justify-content` — aligns items along the **main axis** (e.g. `center`, `space-between`).
- `align-items` — aligns items along the **cross axis** (e.g. `center`, `stretch`).
- `flex-wrap: wrap` — allows items to wrap to the next line.
- `gap` — spacing between flex items.
- On a flex item: `flex: 1` distributes available space equally; `align-self` overrides `align-items`.

```
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 1rem;
  flex-wrap: wrap;
}
```

Listing 13: Flexbox centering example

3.19 CSS Grid

- A **two-dimensional** layout method (rows *and* columns simultaneously).
- `display: grid` — makes the element a grid container.
- `grid-template-columns` — defines column sizes (e.g. `repeat(3, 1fr)`, `200px auto`).
- `grid-template-rows` — defines row sizes.
- `gap` — gutters between rows and columns.
- `grid-column` / `grid-row` — specifies where an item starts and ends.
- `grid-template-areas` — assigns named areas; `grid-area` places items into them.

```
.layout {
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  gap: 1rem;
}

.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main   { grid-area: main; }
.footer { grid-area: footer; }
```

Listing 14: CSS Grid layout example

3.20 CSS Positioning

- `static` — default; element flows normally in the page.
- `relative` — positioned relative to its **normal location**; still in flow.
- `absolute` — removed from flow; positioned relative to the nearest **positioned ancestor**.
- `fixed` — removed from flow; positioned relative to the **viewport**; stays when scrolling.
- `sticky` — toggles between `relative` and `fixed` based on scroll position.

```
header {
  position: sticky;
  top: 0;
  z-index: 100;
  background-color: #111;
}
```

Listing 15: Sticky header example

3.21 Responsive Design and Media Queries

- Responsive design makes a webpage adjust to any screen size.
- **Media queries** apply styles based on device/viewport properties.
- Syntax: `@media (condition) { ... }`
- Common conditions: `max-width`, `min-width`, `orientation`, `prefers-color-scheme`.
- **Mobile-first approach**: write base styles for mobile, then use `min-width` queries to enhance for larger screens.

```css
/* Mobile base styles */
.card { font-size: 14px; padding: 1rem; }

/* Tablet */
@media (min-width: 768px) {
  .card { font-size: 16px; padding: 1.5rem; }
}

/* Desktop */
@media (min-width: 1024px) {
  .card { font-size: 18px; padding: 2rem; }
}
```

Listing 16: Responsive breakpoints (mobile-first)

3.22 CSS Variables (`:root`)

- Define variables on the `:root` selector to make them globally available.
- Variable names start with `--`.
- Access with `var(--variable-name)`.
- Makes styles consistent and easy to update globally.

```css
:root {
  --primary: #7a5af8;
  --bg: #0d0d0d;
  --text: #e8e8e8;
  --radius: 8px;
}

button {
  background-color: var(--primary);
  color: var(--text);
  border-radius: var(--radius);
}
```

Listing 17: CSS variables example

4 JavaScript Fundamentals

JavaScript is the programming language of the web — it adds **behaviour** to HTML and CSS, and (via Node.js) runs on the server too.

4.1 Variables and Data Types

- `var` — function-scoped, hoisted, legacy (avoid in modern code).
- `let` — block-scoped, mutable.
- `const` — block-scoped, **immutable binding** (the value itself may still be mutable for objects/arrays).
- **Primitive types**: `number`, `string`, `boolean`, `null`, `undefined`, `symbol`, `bigint`.
- **Reference types**: `object`, `array`, `function`.

```
let age = 25;            // mutable
const name = "Alice";   // immutable binding
var legacy = true;      // avoid -- function-scoped

// typeof operator
typeof 42;          // "number"
typeof "hello";     // "string"
typeof null;        // "object" (historical quirk!)
typeof undefined;   // "undefined"
typeof [];          // "object"
typeof {};          // "object"
typeof function(){}; // "function"
```

Listing 18: Variable declarations

4.2 Type Coercion

JavaScript is **dynamically typed** and performs implicit type conversion, which can produce surprising results.

```
// Implicit coercion (loose equality ==)
"5" == 5;       // true  (string coerced to number)
0 == false;     // true
null == undefined; // true
null == 0;      // false

// Always prefer strict equality ===
"5" === 5;      // false (no coercion)
0 === false;    // false

// Addition vs concatenation
"5" + 2;        // "52"  (number coerced to string)
"5" - 2;        // 3     (string coerced to number)
"5" * "3";      // 15
+"5";           // 5     (unary + converts string to number)
!!"hello";      // true  (double negation to boolean)

// Falsy values: false, 0, "", null, undefined, NaN
// Everything else is truthy
```

Listing 19: Type coercion examples

4.3 Functions

4.3.1 Hoisting

Hoisting is JavaScript's behaviour of moving declarations to the top of their scope *before* code executes.

- **Function declarations** are fully hoisted — the entire function body is available before its line in the code.

- **var variables** are hoisted but initialised to `undefined` until the assignment line is reached.
- **let and const** are hoisted but **not initialised** — accessing them before their declaration throws a `ReferenceError` (the **Temporal Dead Zone**).
- **Function expressions** and **arrow functions** assigned to variables follow the hoisting rules of `var`/`let`/`const` and are **not** callable before their declaration.

```
// Function declaration -- WORKS (fully hoisted)
console.log(add(2, 3)); // 5
function add(a, b) { return a + b; }

// var -- hoisted as undefined, no error but unexpected value
console.log(x); // undefined (not an error)
var x = 10;
console.log(x); // 10

// let/const -- Temporal Dead Zone (TDZ)
console.log(y); // ReferenceError: Cannot access 'y' before initialization
let y = 20;

// Function expression -- NOT hoisted
console.log(multiply(2, 3)); // TypeError: multiply is not a function
const multiply = (a, b) => a * b;
```

Listing 20: Hoisting behaviour

4.3.2 Function Types

```
// 1. Function declaration (hoisted, has own `this`)
function add(a, b) {
  return a + b;
}

// 2. Function expression (not hoisted, has own `this`)
const subtract = function(a, b) {
  return a - b;
};

// 3. Arrow function (not hoisted, NO own `this`, NO `arguments`)
const multiply = (a, b) => a * b;

// Single parameter -- parens optional
const double = n => n * 2;

// No parameters -- parens required
const greetWorld = () => "Hello, World!";

// Multi-line arrow function -- need explicit return
const divide = (a, b) => {
  if (b === 0) throw new Error("Division by zero");
  return a / b;
};
```

Listing 21: All function declaration styles

4.3.3 Parameters

```
// Default parameters
function greet(name = "World", greeting = "Hello") {
  return `${greeting}, ${name}!`;
}
greet();              // "Hello, World!"
```

```
greet("Alice");       // "Hello, Alice!"
greet("Bob", "Hi");   // "Hi, Bob!"

// Rest parameters (must be last)
function sum(first, ...rest) {
  return rest.reduce((acc, n) => acc + n, first);
}
sum(1, 2, 3, 4); // 10

// Spread to pass array as arguments
const nums = [1, 2, 3];
Math.max(...nums); // 3
```

Listing 22: Default, rest, and spread in functions

4.3.4 Higher-Order Functions

A **higher-order function** is a function that either **takes a function as an argument** or **returns a function** (or both). This is the foundation of functional patterns in JS.

```
// Takes a function as argument
function repeat(n, action) {
  for (let i = 0; i < n; i++) action(i);
}
repeat(3, console.log); // prints 0, 1, 2

// Returns a function (function factory)
function makeMultiplier(factor) {
  return (n) => n * factor; // closure over `factor`
}
const triple = makeMultiplier(3);
triple(5); // 15

// Built-in HOFs: map, filter, reduce, forEach, sort
[1, 2, 3].map(n => n ** 2);       // [1, 4, 9]
[1, 2, 3, 4].filter(n => n % 2 === 0); // [2, 4]
```

Listing 23: Higher-order functions

4.3.5 Immediately Invoked Function Expressions (IIFE)

An **IIFE** is a function that is defined and called immediately. It was commonly used to create a private scope before ES modules existed.

```
// Classic IIFE
(function() {
  const secret = "hidden";
  console.log("IIFE ran, secret:", secret);
})();
// `secret` is not accessible outside

// Arrow IIFE
(() => {
  console.log("Arrow IIFE");
})();

// IIFE with return value
const result = (() => {
  const x = 10, y = 20;
  return x + y;
})();
console.log(result); // 30
```

Listing 24: IIFE pattern

4.4 Objects and JSON

- Objects group related data and behaviour as **key-value pairs**.
- Values can be data (**properties**) or functions (**methods**).
- **JSON** (JavaScript Object Notation) is a text format for storing and exchanging object data.
- Technically, in JavaScript, almost everything is an object.

```
// Object literal
const user = {
  name: "Alice",
  age: 25,
  greet() {
    return `Hi, I'm ${this.name}`;
  },
};

// Property access
user.name;          // dot notation
user["age"];        // bracket notation (useful for dynamic keys)

// Destructuring
const { name, age } = user;

// Spread operator
const updated = { ...user, age: 26 };

// JSON serialisation/deserialisation
const json = JSON.stringify(user);   // '{"name":"Alice","age":25}'
const parsed = JSON.parse(json);     // back to object
```

Listing 25: Objects and JSON

4.5 Arrays

```
const nums = [1, 2, 3, 4, 5];

nums.map(n => n * 2);          // [2, 4, 6, 8, 10] -- transform each element
nums.filter(n => n > 2);       // [3, 4, 5] -- keep matching elements
nums.reduce((acc, n) => acc + n, 0); // 15 -- fold to single value
nums.find(n => n > 3);         // 4 -- first match
nums.findIndex(n => n > 3);    // 3 -- index of first match
nums.some(n => n > 4);         // true -- at least one matches
nums.every(n => n > 0);        // true -- all match
nums.includes(3);              // true
nums.slice(1, 3);              // [2, 3] -- non-mutating
nums.splice(1, 2);             // removes & returns [2, 3], mutates original

// Spread and destructuring
const [first, second, ...rest] = nums;
const combined = [...nums, 6, 7];
```

Listing 26: Array methods

4.6 The `this` Keyword

- Refers to the object that is **calling the function**.
- In **objects**, `this` points to the object itself.
- In **classes**, `this` points to the created instance.
- In **arrow functions**, `this` is **inherited** from the outer (lexical) scope — arrow functions have no own `this`.

- The value of `this` depends on **how** a function is called, not where it is defined.

```
const obj = {
  name: "Alice",
  greet() {
    console.log(this.name); // "Alice" -- this = obj
  },
  greetArrow: () => {
    console.log(this.name); // undefined -- arrow has no own this
  },
};

class Counter {
  count = 0;
  increment() {
    this.count++; // this = instance
  }
}
```

Listing 27: this in different contexts

4.7 Closures

A **closure** is a function that **remembers** the variables from its outer scope even after that scope has finished executing.

```
function makeCounter() {
  let count = 0;        // outer variable
  return function() {   // inner function closes over `count`
    count++;
    return count;
  };
}

const counter = makeCounter();
counter(); // 1
counter(); // 2
counter(); // 3
// `count` is private and persists between calls
```

Listing 28: Closure example

4.8 The Event Loop and Asynchronous JavaScript

JavaScript is **single-threaded** but uses an event loop to handle asynchronous operations without blocking.

- **Call stack**: where currently executing code runs (LIFO).
- **Web APIs**: browser-provided APIs (timers, fetch, DOM events) that run outside the stack.
- **Callback queue** (task queue): completed async callbacks waiting to be run.
- **Microtask queue**: higher-priority queue for Promise callbacks (`.then`, `.catch`).
- The event loop pushes tasks from the queues to the stack only when the stack is **empty**.
- Microtasks always drain before the next macrotask runs.

4.9 Callbacks

- A **callback** is a function passed as an argument to another function, to be called later.
- Simple and flexible, but can lead to deeply nested **"callback hell"**.

```
// Simple callback
setTimeout(() => console.log("Done!"), 1000);

// Callback hell (pyramid of doom)
fetchUser(id, (user) => {
  fetchPosts(user.id, (posts) => {
    fetchComments(posts[0].id, (comments) => {
      console.log(comments); // deeply nested!
    });
  });
});
```

Listing 29: Callback example and callback hell

4.10 Promises

- A **Promise** represents a value that will be available in the future.
- Three states: **pending**, **fulfilled**, **rejected**.
- Chain with `.then()` (success) and `.catch()` (error) and `.finally()`.
- `Promise.all()` — waits for **all** promises; fails fast on any rejection.
- `Promise.allSettled()` — waits for all, regardless of outcome.
- `Promise.race()` — resolves/rejects as soon as the first settles.

```
const fetchData = () =>
  new Promise((resolve, reject) => {
    setTimeout(() => resolve({ id: 1, name: "Alice" }), 500);
  });

fetchData()
  .then(user => { console.log(user.name); return user.id; })
  .then(id => console.log("ID:", id))
  .catch(err => console.error(err))
  .finally(() => console.log("Done"));

// Parallel execution
Promise.all([fetch("/users"), fetch("/posts")])
  .then(([users, posts]) => console.log(users, posts));
```

Listing 30: Promises

4.11 Async / Await

- Built on top of Promises but with **synchronous-looking** syntax.
- `async` before a function makes it return a Promise automatically.
- `await` pauses execution inside the async function until the Promise resolves.
- Use `try/catch` for error handling.
- Easier to read than promise chains; preferred in modern code.

```
async function loadUser(id) {
  try {
    const res = await fetch(`/api/users/${id}`);
    if (!res.ok) throw new Error("User not found");
    const user = await res.json();
    console.log(user.name);
    return user;
  } catch (err) {
```

```
    console.error("Failed:", err.message);
  }
}

// Parallel with async/await
async function loadAll() {
  const [users, posts] = await Promise.all([
    fetch("/users").then(r => r.json()),
    fetch("/posts").then(r => r.json()),
  ]);
  return { users, posts };
}
```

Listing 31: async/await

4.12 ES Modules

- A **module** is a file with its own private scope.
- Use `export` to make values/functions visible to other files.
- Use `import` to use code from other files.
- **Named exports**: `export function foo() {}` — imported as `{ foo }`.
- **Default export**: `export default ...` — imported without braces.
- Modern browsers and Node.js support ES modules natively.

```
// utils.js
export const PI = 3.14159;
export function square(x) { return x * x; }
export default class MathHelper { /* ... */ }

// main.js
import MathHelper, { PI, square } from "./utils.js";
import * as Utils from "./utils.js"; // import all as namespace
```

Listing 32: ES module syntax

5 JavaScript Beyond the Browser

5.1 Limitations of Plain JavaScript

- Originally designed for simple browser interactions, not full-scale applications.
- Single-threaded execution causes blocking during heavy I/O or computation.
- Dynamic typing leads to unpredictable runtime errors in large projects.
- Early JS lacked a proper module system, making code organisation difficult.
- Limited tooling and poor scalability for complex applications.

5.2 Node.js

- Powered by Google's **V8 engine** — the same engine that runs Chrome.
- **Event-driven, non-blocking I/O** model — handles many requests concurrently.
- Same language (**JavaScript**) for both frontend and backend.
- Access to the **npm** ecosystem (millions of packages).
- Turned JavaScript into a complete server-side platform.
- Supports modern ES modules, `async/await`, and integrates with containers and cloud.

5.3 NPM (Node Package Manager)

- Central registry to manage and distribute JavaScript packages.
- Standardises project setup with `package.json`.
- Key commands:
  - `npm init` — create a new project and generate `package.json`.
  - `npm install <package>` — add a dependency.
  - `npm uninstall <package>` — remove a package.
  - `npm update` — update all dependencies.
  - `npm run <script>` — execute a custom script defined in `package.json`.

```
{
  "scripts": {
    "start":  "node server.js",
    "dev":    "nodemon server.js",
    "build":  "tsc",
    "test":   "jest"
  }
}
```

Listing 33: package.json scripts section

5.4 Popular npm Packages

- **Express** — minimalist web framework for APIs and servers.
- **React** — library for building interactive UIs.
- **Lodash** — utility functions for arrays, objects, strings.
- **Axios** — promise-based HTTP client.
- **Socket.io** — real-time bidirectional communication.
- **Jest** — testing framework for JavaScript/Node.

- **Day.js** — lightweight date/time manipulation.
- **Three.js** — 3D graphics in the browser.
- **Electron** — build cross-platform desktop apps with web technologies.

5.5 Bundling for Browsers

- Combines multiple JS files into a single (or few) output files for browsers.
- Reduces HTTP requests and improves load performance.
- Handles module resolution, imports, and exports automatically.
- Often includes **minification** (removing whitespace) and **tree-shaking** (removing unused code).
- Popular tools: **Vite**, **Webpack**, **Rollup**, **Parcel**.

5.6 TypeScript

5.6.1 Motivation

- JavaScript is **dynamically typed** — variables can change type at runtime.
- Type coercion causes surprises: `"5" + 2` evaluates to `"52"`.
- Large codebases are harder to maintain due to hidden type bugs.
- No compile-time checks — many errors appear only during execution.

5.6.2 What is TypeScript?

- Created by **Microsoft** in 2012 (lead architect: **Anders Hejlsberg**).
- A **statically typed superset** of JavaScript — all valid JS is valid TS.
- You write TypeScript → it compiles to JavaScript → runs anywhere JS runs.
- Designed for gradual, optional adoption.
- Widely used in enterprise and complex web applications.

5.6.3 Variables and Types

```
let age: number = 25;
const name: string = "Alice";
let isAdmin: boolean = false;

// Union type
let id: number | string = 42;
id = "abc-123"; // also valid

// TypeScript infers the type if not declared
let score = 100; // inferred as number
```

Listing 34: TypeScript variable annotations

5.6.4 Interfaces

```
interface User {
  id: number;
  name: string;
  email?: string;   // optional
  readonly role: string; // cannot be changed after creation
}

function greet(user: User): string {
  return `Hello, ${user.name}`;
}
```

Listing 35: TypeScript interface

5.6.5 Classes

```
class Animal {
  private name: string;
  protected age: number;

  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }

  public speak(): void {
    console.log(`${this.name} makes a sound.`);
  }
}

class Dog extends Animal {
  constructor(name: string, age: number) {
    super(name, age);
  }

  public speak(): void {
    console.log("Woof!");
  }
}
```

Listing 36: TypeScript class with access modifiers

5.6.6 Generics

```
function identity<T>(arg: T): T {
  return arg;
}

identity<string>("hello"); // returns "hello"
identity<number>(42);      // returns 42

// Generic interface
interface ApiResponse<T> {
  data: T;
  status: number;
  message: string;
}
```

Listing 37: TypeScript generics

5.6.7 Modules

```typescript
// math.ts
export function add(a: number, b: number): number {
  return a + b;
}

export default function multiply(a: number, b: number): number {
  return a * b;
}

// main.ts
import multiply, { add } from "./math";
console.log(add(2, 3));       // 5
console.log(multiply(2, 3));  // 6
```

Listing 38: ES module exports and imports in TypeScript

6 The DOM and Browser APIs

6.1 From Markup to DOM

- The browser **parses HTML** into a tree of nodes — the **DOM (Document Object Model)**.
- Each HTML tag becomes an **element node**.
- Text inside tags becomes **text nodes**.
- Attributes become **node properties**.
- The DOM is what JavaScript can access and modify.

6.2 DOM Structure

- **Root node**: `document`.
- **window** — top-level browser object (global scope).
- **document** — represents the web page inside the window.
- **Elements** — tags (`<div>`, `<p>`) as objects in the tree.
- **Attributes & Text** — properties of element nodes.
- **Events** — methods and listeners that connect JS to user actions.

6.3 Querying DOM Elements

```
// By ID (returns a single Element or null)
const btn = document.getElementById("submit-btn");

// By class name (returns HTMLCollection)
const cards = document.getElementsByClassName("card");

// By tag name (returns HTMLCollection)
const paragraphs = document.getElementsByTagName("p");

// CSS selector -- first match (most flexible)
const nav = document.querySelector("header > nav");

// CSS selector -- all matches (returns NodeList)
const links = document.querySelectorAll("a[href^='https']");
```

Listing 39: DOM querying methods

6.4 Events

- Events let webpages react to user actions.
- Common events: `click`, `input`, `change`, `submit`, `keydown`, `keyup`, `load`, `scroll`, `resize`.
- `addEventListener` is the **modern, preferred** way to attach events.
- Multiple handlers can be added to the same event on the same element.

```
const button = document.querySelector("#myBtn");

button.addEventListener("click", (event) => {
  console.log("Clicked!", event.target);
  event.preventDefault(); // prevent default browser action
});

// Remove listener
button.removeEventListener("click", handlerFunction);
```

Listing 40: Event handling example

6.5 Manipulating the DOM

6.5.1 Modifying Content and Styles

```
const heading = document.querySelector("h1");

heading.textContent = "New Title";         // safe (plain text)
heading.innerHTML = "<em>New</em> Title";  // parses HTML

heading.style.color = "crimson";
heading.style.fontSize = "2rem";

heading.setAttribute("data-active", "true");
heading.getAttribute("data-active"); // "true"
heading.removeAttribute("data-active");
```

Listing 41: Changing content and styles

6.5.2 Adding and Removing Elements

```
// Create and insert
const div = document.createElement("div");
div.classList.add("card");
div.textContent = "Hello!";
document.body.appendChild(div);

// Insert before another element
const list = document.querySelector("ul");
const newItem = document.createElement("li");
newItem.textContent = "New item";
list.insertBefore(newItem, list.firstChild);

// Remove
div.remove();
// or: list.removeChild(newItem);
```

Listing 42: Creating and removing DOM nodes

6.5.3 Working with Classes

```
element.classList.add("active");
element.classList.remove("hidden");
element.classList.toggle("dark-mode");     // adds if absent, removes if present
element.classList.contains("active");      // returns boolean
element.classList.replace("old", "new");
```

Listing 43: classList API

6.6 Forms and DOM

```
const form = document.querySelector("#loginForm");
const emailInput = document.querySelector("#email");

form.addEventListener("submit", (e) => {
  e.preventDefault(); // stop page reload
  const email = emailInput.value;
  const isChecked = document.querySelector("#remember").checked;
  console.log(email, isChecked);
});
```

Listing 44: Reading form input values

6.7 The window Object

- Represents the browser window; all global variables and functions belong to it.
- `alert()`, `confirm()`, `prompt()` — dialogs.
- `location` — current URL; can be used to redirect: `window.location.href = "/new"`.
- `history` — browser history (`back()`, `forward()`).
- `navigator` — browser/device information.
- `setTimeout(fn, ms)` / `setInterval(fn, ms)` — timers.
- `innerWidth` / `innerHeight` — viewport dimensions.
- `scrollX` / `scrollY` — scroll position.

6.8 Web Storage

6.8.1 localStorage

- Stores data **persistently** in the browser (survives closing the tab/browser).
- Data stored as **key-value pairs of strings**.
- Same-origin policy: data is isolated per domain.
- Approximately 5–10 MB per site.
- **Synchronous** and not safe for sensitive information.

```
// Store
localStorage.setItem("theme", "dark");
localStorage.setItem("user", JSON.stringify({ name: "Alice" }));

// Retrieve
const theme = localStorage.getItem("theme"); // "dark"
const user = JSON.parse(localStorage.getItem("user"));

// Remove
localStorage.removeItem("theme");
localStorage.clear(); // removes all keys
```

Listing 45: localStorage usage

6.8.2 sessionStorage

- Data is cleared when the **tab or window is closed**.
- Same API as `localStorage`: `setItem`, `getItem`, `removeItem`, `clear`.
- Useful for temporary session data (e.g. multi-step form progress).

7 React — UIs That React

7.1 Motivation: Problems with Classic Frontend Development

- Tangled DOM manipulation with vanilla JS / jQuery.
- Inconsistent and hard-to-manage **state**.
- Poor code reuse and maintainability.
- Performance bottlenecks from manual DOM updates.
- Difficult collaboration and scaling.

7.2 The Rise and Limits of jQuery

- jQuery made DOM manipulation simpler and faster, and abstracted cross-browser inconsistencies.
- Popularised easy AJAX requests and brought a cleaner API than raw JavaScript.
- However, jQuery still left **state management** and **application structure** problems unsolved.
- As applications grew more complex, jQuery codebases became difficult to maintain and reason about.
- This led to the rise of modern frameworks and libraries like Angular, Vue, and React.

7.3 Library vs. Framework

- **Library**: you call it — you are in control (e.g. React, Axios).
- **Framework**: it calls you — it is in control (e.g. Angular, Next.js).
- Libraries provide tools; frameworks provide structure and enforce conventions.
- **React is a library**, not a framework — it handles the view layer and leaves routing, state, and data-fetching to other tools.

7.4 What is React?

- A **JavaScript library** for building user interfaces (not a full framework).
- Created by **Facebook** in 2013, now maintained by Meta and the open-source community.
- **Component-based**: build UIs from small, reusable pieces.
- **Declarative**: describe *what* the UI should look like, not *how* to update it.
- Efficient rendering via the **Virtual DOM**.

7.5 Setting Up a React Project with Vite

```
npm create vite@latest my-app
# Select "React" or "React + TypeScript" when prompted

cd my-app
npm install
npm run dev
# Opens at http://localhost:5173
```

Listing 46: Creating a React + Vite project

- **Vite** uses native ES modules in development, providing near-instant HMR (Hot Module Replacement).
- Uses **Rollup** for optimised production builds.
- Created by Evan You (Vue.js creator).

7.6 JSX

- A **syntax extension** for JavaScript that lets you write HTML-like code inside JS.
- **Not valid JS** — compiled to plain JavaScript by **Babel**.
- Rules:
  - Must return a **single root element** (use `<>...</>` fragments if needed).
  - Attribute names use **camelCase**: `className`, `onClick`, `htmlFor`.
  - Self-closing tags must close: `<img />`, `<input />`.
  - JavaScript **expressions** go inside {}.
  - Statements (`if`, `for`) are **not** allowed directly inside JSX.

```
const name = "World";
const isLoggedIn = true;

return (
  <div className="container">
    <h1>Hello, {name}!</h1>
    <p>{isLoggedIn ? "Welcome back." : "Please log in."}</p>
    <ul>
      {["Apple", "Banana", "Cherry"].map((fruit, i) => (
        <li key={i}>{fruit}</li>
      ))}
    </ul>
  </div>
);
```

Listing 47: JSX examples

7.7 React Components

- Independent, reusable building blocks of the UI.
- Accept **props** as input and return JSX.
- Two types: **Function-based** (modern, preferred) and Class-based (legacy).
- Component names must start with an **uppercase letter**.

```
interface ButtonProps {
  label: string;
  onClick: () => void;
  disabled?: boolean;
}

function Button({ label, onClick, disabled = false }: ButtonProps) {
  return (
    <button onClick={onClick} disabled={disabled} className="btn">
      {label}
    </button>
  );
}

export default Button;
```

Listing 48: A functional React component

7.8 Props

- **Props** are inputs passed from parent to child components.
- **Read-only**: child components must not modify them.
- Passed like HTML attributes.
- The special `children` prop passes nested JSX into a component.

```tsx
function Card({ title, children }: { title: string; children: React.ReactNode }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      <div className="card-body">{children}</div>
    </div>
  );
}

// Usage:
<Card title="Hello">
  <p>This is the card body content.</p>
</Card>
```

Listing 49: Using props and children

7.9 State and useState

- **State** is component memory that stores dynamic data.
- Changing state **triggers a re-render**.
- Managed with the `useState` hook: `const [value, setValue] = useState(initialValue)`.
- Always update via the setter — **never modify state directly**.
- Use a **callback form** when new state depends on old state: `setValue(v => v + 1)`.

```tsx
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(c => c + 1)}>+</button>
      <button onClick={() => setCount(c => c - 1)}>-</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

Listing 50: useState example — counter

7.10 Event Handling in React

- Use camelCase event names: `onClick`, `onChange`, `onSubmit`.
- Pass a **function reference**, not a function call: `onClick={handleClick}` not `onClick={handleClick()}`.
- Access the event object: `onClick={(e) => console.log(e.target)}`.
- Prevent default behaviour: `e.preventDefault()`.

7.11 Lists in JSX

```tsx
interface Post { id: number; title: string; }

function PostList({ posts }: { posts: Post[] }) {
  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```

Listing 51: Rendering a list with keys

Always provide a stable, unique `key` for list items. Avoid using array index as key when the list can reorder.

7.12 The Virtual DOM and Reconciliation

- React maintains a **Virtual DOM** — a lightweight in-memory copy of the real DOM.
- When state or props change, React builds a **new VDOM tree**.
- The **Fiber reconciler** diffs the new tree against the old one (**O(n)** algorithm with heuristics).
- Only the **minimal set of real DOM updates** is applied.
- Heuristics: (1) Different element types always produce new subtrees; (2) `key` props help track moved elements.

7.13 Component Composition

- Building UIs by **combining** smaller components together.
- Promotes reusability and clean separation of concerns.
- Components can nest inside others: `<Layout><Header /></Layout>`
- Use `props` and `children` to pass content into components.
- Encourages modular, maintainable React architecture.
- Prefer **composition** over inheritance for sharing logic between components.

7.14 React Developer Tools

- **React DevTools** is a browser extension for inspecting React applications.
- Allows inspection of the **component tree** and current props and state.
- Highlights re-renders to help identify performance issues.
- Enables **editing of props and state** during development without code changes.
- Useful for debugging hooks, context, and component hierarchy.
- Also use `console.log()` in key spots to trace state and prop values.

7.15 Styling in React

- **Plain CSS**: import `.css` files; apply with `className`.
- **CSS Modules**: scoped styles per component (filename: `Button.module.css`).
- **Inline styles**: `style={{ backgroundColor: "red", fontSize: "1rem" }}` (camelCase).
- **TailwindCSS**: utility-first classes applied directly in JSX.
- **shadcn/ui**: accessible, copyable React components built on Radix UI + Tailwind.

8 React — Act II: Hooks, Lifecycle, and Patterns

8.1 Component Lifecycle

- **Mount**: component is created and inserted into the DOM.
- **Update**: state or props change, triggering a re-render.
- **Unmount**: component is removed from the DOM.
- Rendering must remain **pure** — same inputs produce the same output; no side effects during render.
- Side effects are managed with `useEffect`.

8.2 The `useEffect` Hook

- Runs **after** the component renders.
- Used to synchronise the component with external systems: APIs, timers, subscriptions, DOM mutations.
- Accepts a **dependency array** that controls when the effect re-runs.
- Can return a **cleanup function** to undo the effect on unmount or before re-running.

```
// Run once on mount
useEffect(() => {
  console.log("Component mounted");
  return () => console.log("Component unmounted"); // cleanup
}, []);

// Run when `userId` changes
useEffect(() => {
  fetchUser(userId).then(setUser);
}, [userId]);

// Run after every render (omit dependency array -- rarely needed)
useEffect(() => {
  document.title = `Count: ${count}`;
});
```

Listing 52: useEffect variations

8.2.1 Common `useEffect` Mistakes

- **Missing dependencies**: causes stale closures and hard-to-debug bugs.
- **Calling `setState` unconditionally inside effects**: can cause infinite re-render loops.
- **Deriving state from props inside effects**: compute it during render instead.
- Effects may run **twice in Strict Mode** (React mounts, unmounts, then mounts again to detect side effects).

8.3 Fetching Data

8.3.1 Using `fetch`

```
import { useState, useEffect } from "react";

function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let cancelled = false;
    setLoading(true);
```

```
    fetch("https://api.example.com/users")
      .then((res) => {
        if (!res.ok) throw new Error("Network error");
        return res.json();
      })
      .then((data) => { if (!cancelled) setUsers(data); })
      .catch((err) => { if (!cancelled) setError(err.message); })
      .finally(() => { if (!cancelled) setLoading(false); });

    return () => { cancelled = true; }; // prevent state update after unmount
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error)   return <p>Error: {error}</p>;
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

Listing 53: Data fetching with fetch + useEffect

8.3.2 Using Axios

```
import axios from "axios";
import { useState, useEffect } from "react";

function UserList() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    const source = axios.CancelToken.source();
    axios.get("/api/users", { cancelToken: source.token })
      .then(({ data }) => setUsers(data))
      .catch((err) => {
        if (!axios.isCancel(err)) console.error(err);
      });
    return () => source.cancel("Component unmounted");
  }, []);

  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

Listing 54: Data fetching with Axios

8.4 State Management Patterns

8.4.1 Pattern I: Lifting State Up

- Move shared state to the **closest common ancestor**.
- Children receive data as props; updates flow upward via callbacks.
- Prevents state divergence between siblings.

8.4.2 Pattern II: Callbacks as Props

- Parent passes a function to a child for state updates.
- The child calls the function to communicate intent, without owning state.
- Enables **unidirectional data flow**.
- May lead to **prop drilling** in deep component trees.

```tsx
function Parent() {
  const [value, setValue] = useState("");
  return <Input value={value} onChange={setValue} />;
}

function Input({ value, onChange }: { value: string; onChange: (v: string) => void }) {
  return (
    <input value={value} onChange={(e) => onChange(e.target.value)} />
  );
}
```

Listing 55: Callback as prop

8.4.3 Pattern III: Context API

- Provides a way to share state **without prop drilling**.
- Data is supplied via a `Provider` and consumed by any descendant via `useContext`.
- Best for **global concerns**: theme, authentication, locale, feature flags.
- **Overusing Context** can cause unnecessary re-renders — scope it tightly.

```tsx
import { createContext, useContext, useState } from "react";

const ThemeContext = createContext<"dark" | "light">("dark");

function App() {
  const [theme, setTheme] = useState<"dark" | "light">("dark");
  return (
    <ThemeContext.Provider value={theme}>
      <button onClick={() => setTheme(t => t === "dark" ? "light" : "dark")}>
        Toggle
      </button>
      <Page />
    </ThemeContext.Provider>
  );
}

function Page() {
  const theme = useContext(ThemeContext);
  return <div className={theme}>Page content</div>;
}
```

Listing 56: Context API example

8.5 Redux

- Predictable state management library for React.
- Application state lives in a **single global store**.
- State updates occur through **dispatched actions** and **pure reducer** functions.
- Enables centralised debugging and **time-travel inspection**.
- Best suited for large applications with complex shared state.

8.6 React Router

- Enables **client-side routing** in SPAs — maps URLs to components without full page reloads.
- Provided by the `react-router-dom` package.
- Core components: `BrowserRouter`, `Routes`, `Route`.

- Hooks: `useParams`, `useNavigate`, `useLocation`.
- Supports nested routes, dynamic segments, and layout composition.

```tsx
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
        <Link to="/users/42">User 42</Link>
      </nav>
      <Routes>
        <Route path="/"          element={<Home />} />
        <Route path="/about"     element={<About />} />
        <Route path="/users/:id" element={<UserDetail />} />
        <Route path="*"          element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Listing 57: React Router v6 example

8.7 Forms in React

- **Controlled components**: form input values are driven by React state; use `value` + `onChange`.
- **Uncontrolled components**: rely on `useRef` to access DOM values directly — simpler but less React-idiomatic.
- Form submission handled via event handlers rather than default browser behaviour.
- Always call `e.preventDefault()` to stop the page from reloading on submit.
- Validation can be synchronous (in state) or asynchronous (with effects or libraries like **React Hook Form**).

```tsx
import { useState } from "react";

function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (!email.includes("@")) {
      setError("Invalid email address");
      return;
    }
    console.log("Submitting:", { email, password });
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />
      <input
        type="password"
        value={password}
```

```
        onChange={(e) => setPassword(e.target.value)}
        placeholder="Password"
      />
      {error && <p style={{ color: "red" }}>{error}</p>}
      <button type="submit">Log In</button>
    </form>
  );
}
```

Listing 58: Controlled form in React

8.8 Error Handling in React

- Errors can occur during **rendering**, **data fetching**, or **user interactions**.
- **Error Boundaries**: class components that catch render-time errors in their subtree and display a fallback UI instead of crashing.
- Async errors (fetch failures, timeouts) must be caught explicitly in `useEffect` or event handlers.
- Runtime errors should always be anticipated and handled gracefully.
- Proper error handling improves resilience and user experience.

```
import { Component, ReactNode } from "react";

interface State { hasError: boolean; }

class ErrorBoundary extends Component<{ children: ReactNode }, State> {
  state: State = { hasError: false };

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  componentDidCatch(error: Error, info: any) {
    console.error("Caught:", error, info);
  }

  render() {
    if (this.state.hasError)
      return <h2>Something went wrong. Please refresh.</h2>;
    return this.props.children;
  }
}

// Usage
<ErrorBoundary>
  <MyComponent />
</ErrorBoundary>
```

Listing 59: Simple Error Boundary class component

8.9 shadcn/ui

- A collection of **reusable, accessible** React components.
- Components are **copied into your project** (not installed as a library dependency) — you own the code.
- Built on top of **Radix UI** (headless, accessible primitives) and **TailwindCSS**.
- Fully customisable since the source lives in your codebase.
- Encourages design consistency without locking into a fixed component library.
- Install individual components: `npx shadcn-ui@latest add button`.

8.10 TanStack Query

- Manages **server/remote state** that can change outside the UI.
- Standardises loading, error, caching, and refetching behaviour.
- Reduces boilerplate compared to manual `useEffect` + `useState`.
- Prevents duplicate requests via **request deduplication**.
- Improves perceived performance via caching and background updates.

```tsx
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
import axios from "axios";

function Users() {
  const { data, isLoading, error } = useQuery({
    queryKey: ["users"],
    queryFn: () => axios.get("/api/users").then(r => r.data),
    staleTime: 60_000, // 1 min before background refetch
  });

  if (isLoading) return <p>Loading...</p>;
  if (error)     return <p>Error!</p>;

  return <ul>{data.map((u: any) => <li key={u.id}>{u.name}</li>)}</ul>;
}

function CreateUser() {
  const queryClient = useQueryClient();
  const mutation = useMutation({
    mutationFn: (newUser: any) => axios.post("/api/users", newUser),
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ["users"] }),
  });

  return (
    <button onClick={() => mutation.mutate({ name: "New User" })}>
      Add User
    </button>
  );
}
```

Listing 60: TanStack Query — useQuery example

8.11 TanStack Mutations

- `useMutation` handles **create / update / delete** operations against the server.
- `onSuccess` handlers typically **invalidate related queries** to refresh cached data.
- **Optimistic updates**: update the UI before the server responds for a snappier feel.
- **Rollbacks**: revert the optimistic update if the server request fails.
- Mutation lifecycle callbacks: `onMutate`, `onSuccess`, `onError`, `onSettled`.

8.12 TanStack Best Practices

- Treat TanStack Query as the default for **API/server state** — use React state or Context for **UI state**.
- Normalise query keys and keep them consistent across the app.
- Prefer **invalidation** over manual refetch wiring for correctness.
- Use `select` / transform patterns to keep components minimal.
- Combine with lightweight client state (Context or Zustand) when needed.

8.13 Next.js and SSR

- **SSR (Server-Side Rendering)**: HTML is rendered on the **server** and sent to the client.
- Improves **initial load performance** and **SEO** (content visible to crawlers immediately).
- Requires **hydration** to attach React interactivity on the client.
- **Next.js**: production-grade React framework with built-in SSR, SSG, file-based routing, and performance optimisations.
- Supports multiple rendering strategies: SSR, SSG (Static Site Generation), CSR (Client-Side Rendering).

8.14 Recommended Project Structure

```
src/
  assets/          # images, icons, fonts
  components/      # reusable UI components (Button, Card, Modal)
  pages/           # route-level components (HomePage, AboutPage)
  hooks/           # custom React hooks (useAuth, useFetch)
  services/        # API calls and external integrations
  context/         # React Context providers
  store/           # Redux / Zustand state
  utils/           # helper functions
  types/           # TypeScript interfaces and types
  App.tsx
  main.tsx
```

Listing 61: Typical React project layout

9 Quick Reference

9.1 JavaScript Cheat Sheet

- **Declarations**: `const` (preferred) > `let` > `var` (avoid).
- **Strict equality**: always use `===` / `!==`; avoid `==` to prevent coercion surprises.
- **Falsy values**: `false`, `0`, `""`, `null`, `undefined`, `NaN`.
- **Array methods**: `.map()`, `.filter()`, `.reduce()`, `.find()`, `.some()`, `.every()`.
- **Object spread**: `{ ...obj, key: value }` creates a shallow copy with overrides.
- **Optional chaining**: `user?.address?.city` — short-circuits to `undefined` instead of throwing.
- **Nullish coalescing**: `value ?? "default"` — uses right side only when left is `null`/`undefined`.
- **Async**: Callbacks → Promises → `async/await` (cleanest).
- **Modules**: `export` / `import`; default vs. named exports.

9.2 HTML Cheat Sheet

- `<!DOCTYPE html>` — HTML5 declaration.
- `<html>`, `<head>`, `<body>` — document skeleton.
- `<h1>`–`<h6>`, `<p>`, `<strong>`, `<em>` — text.
- `<a href>`, `<img src alt>`, `<video>`, `<audio>` — links and media.
- `<ul>`, `<ol>`, `<li>`, `<dl>`, `<dt>`, `<dd>` — lists.
- `<table>`, `<tr>`, `<th>`, `<td>` — tables.
- `<form>`, `<input>`, `<select>`, `<textarea>`, `<button>` — forms.
- `<header>`, `<nav>`, `<main>`, `<section>`, `<footer>` — semantics.

9.3 CSS Cheat Sheet

- Selectors: `element`, `.class`, `#id`, `A > B`, `A + B`, `:hover`, `::before`.
- Box model: `margin` → `border` → `padding` → `content`.
- Layout: `display: flex | grid | block | inline`.
- Position: `static | relative | absolute | fixed | sticky`.
- Responsive: `@media (min-width: 768px) { ... }`.
- Variables: `--name: value;` and `var(--name)`.

9.4 React Hooks Summary

- `useState(init)` — local component state; triggers re-render on change.
- `useEffect(fn, deps)` — side effects (fetch, subscriptions, timers); runs after render.
- `useContext(Context)` — consume a context value provided by a `Provider` ancestor.
- `useRef(init)` — mutable ref that persists across renders without causing re-renders; also used for DOM access.
- `useMemo(fn, deps)` — memoised computed value; re-computes only when dependencies change.
- `useCallback(fn, deps)` — memoised function reference; prevents unnecessary child re-renders.
- `useReducer(reducer, init)` — complex state logic with actions (like a mini Redux).
- `useLayoutEffect(fn, deps)` — like `useEffect` but fires **synchronously** before the browser paints.
- **Rules of Hooks**: only call at the **top level** (not inside loops/conditions), only in **React function** components or custom hooks.